Pretraining

Lecture 2: data, training and scale

Aarne Talman

29 October 2026

LDA-T318 | LLMs and Generative AI for NLP | University of Helsinki

Course overview

Wk	Dates	Tuesday	Thursday	Teacher
1	**27 / 29 Oct**	**Language models**	**Pretraining**	**Aarne**
2	3 / 5 Nov	Post-training	Lab 2: tune vs prompt	Aarne
3	10 / 12 Nov	Inference	Lab 3: serve	Aarne
4	17 / 19 Nov	Evaluation	Lab 4: evaluate	Jussi
5	24 / 26 Nov	RAG	Lab 5: RAG	Dmitry
6	1 / 3 Dec	Agents	Lab 6: agent	Dmitry
7	8 / 10 Dec	Course recap	Special topics (TBC)	All

Reusing a task and model across labs is recommended, not required.

There are no intermediate submissions; the final report and code archive determine the grade.

Lecture overview

1. The training loop *8 min*
2. Data *25 min*
3. Scaling laws *10 min*
4. Training at scale *7 min*
5. Mid-training *10 min*
6. What pretraining produces *10 min*
7. Lab 1 briefing *8 min*

Take away

By the end of today you should be able to explain how a language model is pretrained, make a rough compute estimate, and identify the main choices in its data recipe.

The training loop

What does pretraining ask the model to do?

$$\mathcal{L}(\theta) = -\frac{1}{T}\sum_{t=1}^{T} \log p_\theta(x_t \mid x_{<t})$$

In words: at every position, the model predicts the next token. It receives a larger penalty when it assigns a low probability to the token that actually follows. Here x_t is the token to predict, $x_{<t}$ is the preceding text, and p_θ is the probability assigned by the model.

During training, the model sees the **true preceding tokens**. Every position is therefore a training example, and all positions in a sequence can be processed in parallel.

A sequence of 4096 tokens provides thousands of prediction examples in one pass through the model.

Take away

No manual labels are needed: the text supplies both the input and the correct next token.

One training step

1. **Predict.** Run a batch of text through the model and predict the next token at every position.
2. **Measure the error.** Compare the predictions with the actual next tokens and calculate one average loss.
3. **Work out what to change.** Backpropagation measures how each model weight contributed to that loss.
4. **Update the model.** The optimiser makes a small adjustment to the weights.

Repeat these steps over many batches. If training is working, the model gradually assigns more probability to the tokens that occur in its data.

Why transformers made large-scale training practical

Predictions for all positions in a training sequence can be computed in parallel. An RNN (recurrent neural network) must process the sequence one position at a time.

A BATCH CONTAINS MANY TOKEN PREDICTIONS

A **batch** is the text processed before one model update. In language-model training, its size is most clearly reported as a number of tokens.

$$32 \text{ sequences} \times 2048 \text{ tokens} = 65{,}536 \text{ tokens per step}$$

Why use batches?

- many predictions can run in parallel on a GPU
- averaging over examples gives a more reliable update

What limits their size?

- longer sequences and larger batches use more memory
- the practical choice depends on the available hardware

Why this matters: token batch size connects the training code to GPU use and to the total number of tokens seen during a run.

How do we know whether training is working?

Training loss

- should fall quickly at first, then more slowly
- shows how well the model predicts its training data

Validation loss

- is measured on text excluded from training
- should usually follow the same downward trend
- a growing gap can indicate memorisation

What the curves are used for

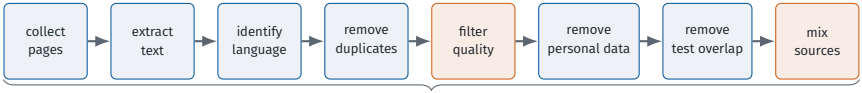
They help us check that the run is learning, compare saved versions from the same run, and notice data or configuration problems early.

Compare like with like

Loss values are directly comparable only when the models use the same tokenizer and comparable data. A lower loss across different setups does not automatically mean a better language model.

Data

The pipeline



The raw crawl is not yet suitable for training. Each stage removes or transforms some material, but every decision can also remove useful text. The right choices depend on the languages and tasks the model should support.

Take away

Two models with the same architecture can learn different things because their training data was prepared differently.

Stage 1: collect web pages

A **web crawl** is a large collection of downloaded web pages. **Common Crawl** is a non-profit project that has collected snapshots of the web roughly monthly since 2008. Each snapshot contains billions of pages.

It provides two useful versions of each page:

- **WARC**: the original response, including HTML and headers
- **WET**: plain text extracted by Common Crawl

WET is easier to use. WARC gives you control over text extraction and usually produces cleaner training data.

What can go wrong?

If you use WET, you also accept Common Crawl's decisions about which parts of the page count as text. Navigation bars, cookie notices and link lists may remain. Recent datasets such as FineWeb, RefinedWeb, Dolma and HPLT therefore start from **WARC** and extract the text themselves. The crawl is also not a representative sample of all writing. Public, easily downloaded pages are over-represented; paywalled and login-protected text is under-represented.

Source: CommonCrawl; Penedo et al., *The FineWeb Datasets*, 2024; Penedo et al., *RefinedWeb*, 2023.

Stage 2: text extraction

Goal: turn an HTML page into clean text in the correct reading order.

Remove: navigation, sidebars, cookie notices, “related articles”, comment spam, licence footers, JavaScript.

Keep: the body text, in reading order, with paragraph and list structure intact.

trafilatura is one widely used extraction tool. FineWeb compared Common Crawl’s WET text with trafilatura applied to WARC pages. Models trained on the latter scored better, but extracting the text required more time and computing resources.

What can go wrong?

Extraction errors are often **systematic**: the same mistake is repeated on every page using a particular website template. For example, an extractor might remove the first paragraph from every page on a large content-management system. A small random sample may not reveal the problem because the remaining text still looks plausible.

Source: Penedo et al., *The FineWeb Datasets*, 2024; Barbaresi, *trafilatura*, 2021.

Stage 3: language identification

A language classifier, often fastText, predicts a language and a confidence score for each document. The pipeline keeps a document only if the score is above a chosen threshold.

Problem	What happens
short documents	unreliable under about 20 words, so short texts vanish
code-switching	a Finnish post quoting English clears neither threshold
related languages	Danish/Bokmål and Czech/Slovak get confused
threshold choice	a higher threshold gives cleaner but less data

FineWeb used a confidence threshold of 0.65 for English. A smaller language may need a lower threshold because much less text is available.

Source: Joulin et al., fastText, 2016; Penedo et al., *The FineWeb Datasets*, 2024.

Stage 4: remove duplicate text

Pipelines usually remove duplicates at three levels:

- **exact duplicates**: identical documents, such as mirrors and reposts
- **near-duplicates**: documents that are mostly the same, such as templated or lightly edited pages
- **repeated lines**: boilerplate that appears across many pages, such as menus and legal notices

Repeated text receives more weight during training simply because it occurs more often. Removing duplicates reduces unwanted memorisation and prevents boilerplate from dominating the corpus.

More deduplication is not always better

FineWeb obtained **worse** models when it removed duplicates across all 96 Common Crawl snapshots than when it processed each snapshot separately. Useful text is often copied across sites or remains online for years. Removing every repeated copy can therefore remove a useful signal of quality while retaining unique, low-quality pages. The deduplication strategy must be tested, not assumed.

Source: Lee et al., *Deduplicating Training Data Makes LMs Better*, 2022; Penedo et al., *The FineWeb Datasets*, 2024.

Stage 5a: rule-based quality filtering

Many web corpora use simple rules to decide whether to **keep or discard each document**. Two influential examples are **C4**, the web corpus used to train T5, and the data pipeline described for DeepMind's **Gopher** language model.

Example criteria for keeping a document

- its length is between 50 and 100 000 words
- the mean word length is 3–10 characters
- at least 80% of words contain a letter
- most lines end with sentence punctuation
- fewer than 30% of lines end in an ellipsis
- it has few symbols and no blocked terms

What are the rules trying to remove?

Pages dominated by menus, code, random symbols, spam or other text that does not resemble ordinary prose. But the rules are often **English-shaped**.

Word-length criteria can penalize Finnish or German compounds, while punctuation criteria may not fit Thai. Blocklists can also remove legitimate text, including sex education and writing by or about minority groups.

Exact rules differ between pipelines. They are cheap and auditable, but they can discard useful text and keep polished-looking spam.

Source: Raffel et al., *C4*, 2020; Rae et al., *Gopher*, 2021; Dodge et al., *Documenting the English C4*, 2021.

Stage 5b: quality filtering by classifier

A classifier can learn which documents to keep. **FineWeb-Edu** provides one example:

1. A large language model rates 460,000 documents from 0 to 5 for *educational value*.
2. Train a much smaller classifier to imitate those ratings.
3. Apply it to all 15 trillion tokens in FineWeb.
4. Keep documents with a score of 3 or higher.

Result: **1.3 trillion tokens** remain. With the same training budget, this subset produces better results on knowledge and reasoning benchmarks.

The trade-off

The model that produced the ratings now defines "good data". "Educational value" favours textbooks, tutorials and encyclopaedic prose over dialogue, fiction and informal writing. This may improve knowledge benchmarks but make the corpus less useful for conversational or creative tasks.

Source: Lozhkov et al., *FineWeb-Edu*, 2024; Penedo et al., *The FineWeb Datasets*, 2024.

How do we know which pipeline is better?

Looking at samples is not enough

Samples and corpus statistics reveal obvious problems. The stronger test is to train comparable small models on data from each pipeline and evaluate them on the same tasks.

A controlled comparison

- fix the architecture, tokenizer, training settings and total number of training tokens
- train a 1.8B-parameter model on 28B tokens from each candidate dataset
- evaluate on a fixed suite of benchmarks chosen in advance for low variance and a signal above chance at that scale
- change **one** pipeline decision at a time

Source: Penedo et al., *The FineWeb Datasets*, 2024.

Why train a model to compare data?

Two datasets can both look reasonable to a person yet teach a model different things. This method isolates the effect of one data-processing decision. It is also expensive: every filter you want to test requires another training run.

Stage 6: remove personal information (PII)

Personally identifiable information (PII) includes email addresses, phone numbers, IP addresses, postal addresses and identity numbers.

Pattern matching finds structured details; named-entity recognition finds names and addresses. Detected text can be replaced with a placeholder so the rest of the sentence remains usable.

This matters for two reasons:

- **legal**: GDPR requires a lawful basis, data minimisation and appropriate safeguards
- **model behaviour**: a model may memorise and reproduce contact details from its training data

Source: Soldaini et al., *Dolma*, 2024; FineWeb-PII, 2024.

What can go wrong?

The tools can miss personal information and remove text that is not personal information. They may miss obfuscated details, mistake version numbers for IP addresses, or mistake a Finnish place name for a person's name. Removing obvious PII reduces risk, but it does not guarantee privacy. Context can still reveal a person's identity.

Stage 7: remove overlap with test data

Decontamination removes benchmark or test examples from the training corpus.

How: compare training documents with each test set. An n-gram is a sequence of n tokens. A long match suggests copying, so the matching passage or document is removed.

Why it is hard

- benchmarks must be known before training
- paraphrases and translations may not share the same words
- future benchmarks or later training can introduce new overlap

Source: Brown et al. 2020; Soldaini et al., *Dolma*, 2024.

Why it matters

If a model saw test questions during training, its score may measure memory rather than generalisation. In your project, keep the test split separate from prompt design, training and model selection. Use it only for the final evaluation.

Stage 8: choose the data mixture

The final step decides how much data to draw from each source. This strongly affects the skills and knowledge the model develops.

Source	Illustrative share	Why include it?
filtered web	60–85%	broad coverage and varied writing styles
code	5–20%	programming and structured problems
maths	2–10%	arithmetic and multi-step reasoning
books, papers	2–10%	long documents and formal writing
curated reference	1–5%	reliable facts and definitions

These shares vary considerably between models. **Upsampling** means drawing from a small source more often, so the model sees some documents more than once. Moderate repetition can help; too much can cause memorisation and reduce the value of further training.

Source: Muennighoff et al., *Scaling Data-Constrained Language Models*, 2023; Aryabumi et al., *To Code or Not To Code?*, 2024.

A SMALLER LANGUAGE HAS A SMALLER DATA BUDGET

- English is roughly **45%** of CommonCrawl documents; Finnish roughly **0.2%**
- after filtering, usable Finnish web text is on the order of **tens of billions** of tokens

A compute-optimal 7B model uses about 140B tokens, and many recent recipes use substantially more. Finnish data alone is therefore not enough for this kind of general-purpose model.

The consequence

A practical approach is a **multilingual** model in which Finnish is upsampled. This supplies enough varied training text while giving Finnish a larger share than its raw web frequency. It can also transfer useful patterns learned from English and code to Finnish contexts.

Source: CommonCrawl language statistics; TurkuNLP Finnish web corpora; Luukkonen et al., *Poro 34B*, 2024.

Ways to increase training data for a smaller language

Approach	What it provides	Main limitation
multilingual mixture	more total data and cross-lingual transfer	less compute spent directly on the target language
repeat scarce data	greater weight for existing target-language text	diminishing returns and more memorisation
machine translation	large volumes from high-resource languages	translation patterns and MT errors
model-generated text	controllable topics and formats	generator bias, errors and licence constraints
parallel text (translations)	the same content in multiple languages	limited volume and domain coverage

These approaches are usually combined. Poro, for example, mixed Finnish with English and code, repeated Finnish data, and included aligned English–Finnish text.

Source: Muennighoff et al. 2023; Luukkonen et al., *Poro 34B*, 2024.

Case study: Poro and Viking on LUMI

Poro 34B (2024) — Silo AI with TurkuNLP and HPLT, trained on LUMI on roughly **1T tokens**:

- English (SlimPajama), the majority of the mixture
- *all* the usable Finnish there was — on the order of 30B tokens, repeated several epochs
- a large code share (StarCoder)
- explicit **English–Finnish translation pairs**

Viking 7B / 13B / 33B — the follow-up: about 2T tokens across Finnish, English, Swedish, Danish, Norwegian, Icelandic and code.

What it bought

Finnish performance far beyond anything trainable on Finnish alone, and English did not suffer for it. That is the cross-lingual transfer argument, demonstrated at scale on a national supercomputer.

What I would do differently

The architecture was frozen roughly nine months before release, and the field moved. An amount of training data that looked ambitious at kick-off was unremarkable at launch. And a large share of the engineering effort went into the software stack rather than the model.

Source: Luukkonen et al., *Poro 34B and the Blessing of Multilinguality*, 2024; Viking model cards, Silo AI / TurkuNLP / HPLT, 2024. Mixture shares approximate.

Licensing, and the open-data question

What is actually released

- **weights only** — almost everything called “open”: Llama, Qwen, Gemma, DeepSeek, Mistral
- **weights + data + code + recipe** — OLMo, and very little else
- **data only** — FineWeb, Dolma, HPLT, The Stack

The legal status of training on copyrighted web text is unsettled and differs between the EU and the US. The EU’s text-and-data-mining exception has an opt-out that publishers increasingly exercise.

Why this matters for research

If the data is secret, the experiment is not reproducible, and claims about architecture cannot be separated from claims about data. That is the concrete reason I said on Tuesday that “which architecture is best” is unanswerable from public evidence.

For your project: name the licence of every model and dataset you use, and check that it permits academic use. This is in the rubric.

Take away

At the same scale, what separates two models is mostly what they were trained on, and that is the part of a recipe least likely to be documented.

Scaling laws

What scaling laws tell us

Across many experiments, three broad patterns appear:

1. a larger model can learn more complex patterns;
2. more training data gives the model more examples to learn from;
3. more compute usually lowers validation loss, but each additional improvement costs more than the previous one.

These trends are called **scaling laws**. They are fitted from smaller training runs and used to estimate what may happen at a larger scale.

Why this is useful

Before paying for a large run, a team can compare possible model sizes, data budgets and expected loss using smaller experiments.

Use them as forecasts, not guarantees about a model or task.

Source: Kaplan et al., *Scaling Laws for Neural Language Models*, 2020.

What the Chinchilla result estimates

Question: with a fixed training-compute budget, what combination of model size and training data is expected to give the lowest validation loss?

What the study found

A very large model trained on too little text is undertrained. A very small model cannot make full use of a very large dataset.

For a fixed compute budget, the study estimated a balance between the number of parameters and the number of training tokens.

A concrete comparison

- Chinchilla: 70B parameters trained on 1.4T tokens
- earlier Gopher: 280B parameters trained on 300B tokens
- similar compute; the smaller Chinchilla performed better

The estimate was about **20 training tokens per parameter**: for example, roughly 140B tokens for a 7B model.

This is a 2022 estimate for particular models and data, not a universal rule.

Source: Hoffmann et al., *Training Compute-Optimal Large Language Models*, 2022.

A SIMPLE ESTIMATE OF TRAINING COMPUTE

For a standard dense transformer:

$$\boxed{\text{training operations} \approx 6 \times \text{parameters} \times \text{training tokens}}$$

The factor 6 is a useful approximation for the forward and backward passes. This relationship is often written as $C \approx 6ND$.

For a **20-million-parameter** Lab 1 model trained on **1 billion tokens**:

$$6 \times 20 \text{ million} \times 1 \text{ billion} = 120 \text{ quadrillion operations}$$

What the estimate is used for

Compare the approximate compute required by different model sizes and training-data budgets before running the experiments.

One FLOP is one floating-point arithmetic operation. This estimate does not predict loss or wall-clock time; it only estimates compute.

How a scaling law predicts validation loss

Fit the relationship

1. Train several smaller models with different parameter and token counts.
2. Measure validation loss on the same held-out data.
3. Fit a smooth curve relating model size and data size to loss.
4. Insert the proposed larger model and data sizes into that curve.

What you get

A **numerical estimate** of expected validation loss, not only the direction of change. It is an extrapolation with uncertainty. It is most credible when the data, tokenizer, architecture and training recipe remain comparable. It does not directly predict task accuracy, reliability or usefulness.

Take away

$6ND$ estimates compute; a fitted scaling law estimates loss. Neither replaces downstream evaluation.

Training at scale

Why training uses much more memory than inference

During inference, memory is used mainly for the model and the current input. Training must also keep information needed to learn from each batch:

- **parameters**: the model's learned numbers;
- **gradients**: signals showing how those numbers should change;
- **optimiser state**: running statistics used when updating them;
- **activations**: intermediate values needed to calculate gradients.

As a rough example, a 7B-parameter model may need more than **100 GB** before all activation memory is counted. Exact requirements depend on the training setup.

Take away

A model that fits on one GPU for inference may still require several GPUs to train.

What changes when training spans many GPUs?

The work has to be divided across devices:

- different GPUs process different data or different parts of the model;
- they regularly exchange results over the network;
- adding GPUs reduces elapsed time only while communication remains fast enough.

Why this matters: scaling is a systems problem as well as a modelling problem.

Long runs also need recovery

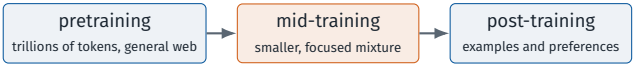
Large jobs run for days or months, so hardware and software failures are expected. The training system saves **checkpoints**: periodic copies of the model and training state from which a run can resume.

During Llama 3 405B training, automated recovery allowed the system to spend over 90% of its allocated time doing useful training despite hundreds of interruptions.

Source: Grattafiori et al., *The Llama 3 Herd of Models*, 2024; personal experience, Poro / Viking on LUMI, 2023–2024.

Mid-training

Mid-training: targeted continued pretraining



Same objective, more focused data. The model still learns by predicting the next token, but it now sees a smaller mixture selected for particular languages, domains or capabilities.

Why it exists. It is cheaper to emphasise specialised data near the end than to give it the same emphasis throughout the full pretraining run.

Many papers call this **continued pretraining**; "mid-training" is a newer name for the same general idea.

Source: OLMo 2 (Allen AI, 2024) is the clearest publicly documented example; Raschka, *The State of LLMs*, 2025, on the terminology.

A short final stage with higher-quality data

After broad pretraining, training can continue briefly on a smaller, carefully selected mixture.

The mixture may emphasise:

- high-quality explanatory text;
- mathematics or code;
- long documents;
- a target language or domain.

Why do this near the end?

It is cheaper than repeating the full pretraining run, and the same base checkpoint can be continued with different mixtures for different goals.

The trade-off

Focusing too strongly on one domain can reduce performance elsewhere, so the resulting model still needs broad evaluation.

Source: OLMo 2 (Allen AI, 2024).

What targeted data can add

A mid-training mixture is selected according to what the model should improve:

Type of data	Why include it?
mathematics and code	more practice with structured, multi-step problems
long documents	practice using information across longer inputs
target language or domain	better coverage of under-represented text
synthetic data	model-generated examples when human-written data is scarce

A focused mixture can improve its target area while reducing performance elsewhere. Evaluate both the target capability and broader model quality.

Source: OLMo 2 (2024); Aryabumi et al., *To Code or Not To Code?*, 2024; Gunasekar et al., *Textbooks Are All You Need*, 2023.

What pretraining produces

In-context learning, live

Demo: a base model that has not been trained to act as a chat assistant.

```
Review: The food was cold and the waiter ignored us.  Sentiment: negative
Review: Beautiful room, spotless, and the staff were kind.  Sentiment: positive
Review: It arrived late and the box was crushed.  Sentiment: negative
Review: Exactly what I wanted, and cheaper than the shop.  Sentiment:
```

The model completes `positive`. Its parameters are not updated for this task.

The examples in the prompt show both the task and the required answer format. The most likely continuation is therefore the correct label.

In-context learning means using examples in the prompt to perform a task without updating the model's parameters.

Source: Brown et al., *Language Models are Few-Shot Learners*, 2020.

What in-context learning means

What happens. The model uses the text already in the prompt when predicting what comes next. Its parameters do not change, and the effect disappears when the prompt is removed.

What the examples provide

- a description of the task;
- examples of inputs and outputs; and
- the expected answer format.

Results can be sensitive to which examples are chosen and how the prompt is formatted.

Why it emerges at all

The pretraining corpus is full of documents in which a pattern is established and then continued: glossaries, tables, translation pairs, exercise-and-answer sets, code with consistent conventions. Next-token prediction therefore rewards models that can detect a pattern in the beginning of a document and continue it.

As an optional Lab 1 extension, run the same few-shot prompt on your tiny model and a larger base model. Treat this as a qualitative comparison: many things besides scale differ.

Source: Brown et al. 2020; Min et al., *Rethinking the Role of Demonstrations*, 2022.

Now ask it a question

Demo: the same base model, now given a direct question without examples.

```
Prompt:  What is the capital of Finland?

Output:  What is the capital of Sweden?
         What is the capital of Norway?
         What is the largest city in Finland?
         What is the population of Helsinki?
         Answer the following questions in complete sentences.
         1. What is the capital of Finland? _____________
         2. What is ...
```

Nothing has necessarily gone wrong here. More questions are one plausible continuation because question lists and worksheets occur in the pretraining corpus.

Why does it keep going? Pretraining did not teach a special “assistant has finished its turn” convention. It only taught the model to continue documents.

What the base model does not have

Missing	Why pretraining could not supply it
reliable stopping	web documents do not mark the end of an assistant turn
turn-taking	ordinary web text does not label user and assistant roles
instruction following	instructions occur, but obeying them is rarely the likeliest continuation
a reliable answer	good and bad answers appear at their web frequencies
refusal	declining a request is rare, and inconsistent where it occurs
a consistent persona	the corpus is millions of different authors

A base model may contain useful knowledge and patterns without consistently producing the behaviour we want. Post-training makes desired responses more likely.

Source: Ouyang et al., *InstructGPT*, 2022, for the original framing.

What pretraining gives us

What pretraining produced

- a statistical model of patterns in a large text collection
- broad knowledge and patterns of language, code and reasoning
- the ability to use examples in a prompt
- a starting point that can be adapted with much less data than pretraining required

What it did not produce

- reliable assistant-style turn-taking, instruction following or stopping behaviour

Why base models are still released

A base model is a useful starting point when researchers want to study or adapt the model before assistant behaviour has been added. Lab 2 compares a base model with its assistant-tuned version, often called an **instruct model**. You may use a different suitable pair from the one used elsewhere in your project.

Take away

Pretraining supplies broad language and task capability. Post-training can select, strengthen and sometimes add behaviours using much less compute, but it still depends on what the base model can represent.

Lab 1 briefing

Lab 1: from tokenizer to tiny GPT

Core A: compare tokenizers

Train 8k and 32k byte-level BPE tokenizers on Finnish Wikipedia. Compare tokens per word on Finnish and English text.

Core B: train one tiny GPT

Run the supplied training configuration, plot training and validation loss, and record model size, tokens, runtime and estimated compute.

Explain the result

State whether the model learned, what evidence supports that conclusion, and what this small experiment cannot show.

Optional extensions

- compare a third language or vocabulary size
- compare three model sizes trained on the same number of tokens
- compare your tiny model with a pretrained base model
- draft a plan for a task you may use in later labs

Start with `lab1-example/README.md`; it includes a CPU smoke test, the training script, configurations and plotting code.

There is no intermediate submission. Keep the code, configuration, results and analysis for the final report.

How to check that a run is working

Check	Expected pattern	If it differs
initial loss	near log vocabulary size; about 10.4 for 32k tokens	check vocabulary and target tokens
training and validation loss	both fall; validation should not begin rising	inspect batches, the data split and possible overfitting
tokens processed per second	becomes stable after the first steps	inspect data loading and GPU use
output files	metrics, summary and model file exist	read the end of the job log

Why the first loss is predictable: before learning, the model assigns roughly equal probability to all V vocabulary items. Its loss should therefore be near $\ln(V)$; for 32,000 tokens this is about 10.4.

Material for the final report

1. a Finnish–English **tokens-per-word table** for the 8k and 32k tokenizers
2. the **training and validation loss curves** for one model
3. model size, tokens processed, runtime and estimated compute
4. a short interpretation of what the experiment shows and does not show
5. the scripts, configuration and result files needed to reproduce it

What makes the analysis useful

A number with the method that produced it, and a sentence saying what you concluded from it. If a result surprises you, investigate it and record what you checked. “The three runs did not order the way I expected, and here is what I checked” is a good paragraph.

Scaling and pretrained-model comparisons belong in the report only if you complete those optional extensions. There is no separate Lab 1 submission.

Working on Lab 1

- Roihu is down for maintenance on 3–4 November. Those are not available for GPU work.
- **There is no supervised lab this week.** The next time we are all in a room with GPUs is 5 November.
- Questions go to the course channel. It is watched all week and answers are visible to everybody.
- There is no intermediate submission. Keep your work for the final report and code archive.

If something breaks

Post the **job id**, the **command**, and the **last twenty lines** of the log. Three quarters of problems are solved from that alone. Do not post screenshots of a terminal.

A useful starting point

Run the bundled CPU smoke test, then Part A. This checks your environment and data flow before you request GPU time.

Three things to remember

1. **Training compute grows with both model size and data size.** $C \approx 6ND$ is a rough way to compare proposed runs, not a prediction of runtime or model quality.
2. **Data choices strongly affect model quality.** Inspect the data for obvious problems, then compare pipeline choices by training small, controlled models and evaluating them.
3. **Pretraining creates a base model, not an assistant.** It learns to continue text and can use examples in a prompt, but reliable instructions, turn-taking and stopping require later training.

Tuesday: the model that will not stop talking

We ended on a base model that answered a question with more questions and then kept going.

This is the gap between a model of *documents* and an *assistant*. Post-training uses examples of desired responses and human or automated feedback to make useful assistant behaviour more likely.

Tuesday 3 November

Post-training: how conversation data represents user and assistant turns, how training on example responses teaches desired behaviour, and how preference feedback shapes it further.

The base-versus-assistant-tuned demo will be pre-recorded, because Roihu is down that day.

This week: begin Lab 1 and keep the outputs for your final report. There is no intermediate submission.

Reading

Core

- Stanford CS336, *Language Modeling from Scratch*, the lectures on data and on systems
- Hugging Face, *The Smol Training Playbook*, 2025
- Karpathy, *nanoGPT*

Data

- Penedo et al., *The FineWeb Datasets*, 2024
- Lozhkov et al., *FineWeb-Edu*, 2024
- Soldaini et al., *Dolma*, 2024
- Luukkonen et al., *Poro 34B and the Blessing of Multilinguality*, 2024

Scaling

- Kaplan et al. 2020; Hoffmann et al. 2022 (Chinchilla)
- Muennighoff et al., *Scaling Data-Constrained Language Models*, 2023
- Schaeffer et al., *Are Emergent Abilities a Mirage?*, 2023

Systems

- Rajbhandari et al., *ZeRO*, 2020
- Shoeybi et al., *Megatron-LM*, 2019
- Grattafiori et al., *The Llama 3 Herd of Models*, 2024; read section 3 on infrastructure
- DeepSeek-AI, *DeepSeek-V3 Technical Report*, 2024, for fp8 training in detail

For current developments, Raschka's *Ahead of AI*: *A Dream of Spring for Open-Weight LLMs* (Feb 2026) and *The State of LLMs* (2025).